

Bayesian Clustering with User-Side Information for Streaming Recommender Systems: Addressing Data Sparsity and Cold-Start with Provable Convergence

July 3, 2026

Abstract

Collaborative filtering, a popular technique in recommendation systems, faces primary challenges: data sparsity, cold-start limitations, and an inability to adapt to streaming data in real-world environments. Although existing heuristic and distance-based approaches partially address streaming scenarios, they lack theoretical rigor and fail to leverage evolving user-item interactions effectively. This study introduces a novel Bayesian clustering framework that integrates user-side information with probabilistic priors to overcome these challenges. The proposed clustering method reduces the impact of rating matrix sparsity by leveraging prior knowledge for streaming data. Furthermore, incorporating user-side information mitigate the effects of the cold-start problem on prediction accuracy. The convergence of the proposed clustering method is established in this study through the proof that the Hessian matrix of its objective function is positive definite. Empirical validation demonstrates a faster convergence rate of proposed method compared to the Fuzzy C-Means. The experimental results on real-world streaming datasets (MovieLens, yelp) indicate that the proposed method, in addition to overcoming the limitations, achieves higher accuracy in predicting user ratings than other methods. By integrating Bayesian inference with the ability to adapt to streaming data, this study addresses key limitations in current recommender systems research.

Keywords: Bayesian Clustering, Streaming Data, Collaborative Filtering, Cold-Start Mitigation, Data Sparsity, Convergence Analysis, Side Information, Recommendation Systems.

1 Introduction

Every piece of knowledge builds upon another, and the understanding of any truth requires awareness of its foundation. Recommender systems, much like the evolution of knowledge, build upon prior information to refine and enhance decision-making. These systems aim to maximize user satisfaction by generating a list of the most relevant and personalized items. Given their significance, extensive research has been conducted in this domain, exploring various methodologies to improve recommendation quality. Each approach relies on different sources of information, benefiting from past interactions to predict future preferences. However, much like gaps in historical knowledge challenge understanding, recommender systems also struggle with incomplete data, cold-start issues, and the dynamic nature of evolving user-item interactions.

Recommender systems aim to enhance user satisfaction by generating a list of the most relevant and personalized items. Given their significance, extensive research has been conducted in this domain, exploring various methodologies for improving recommendation quality. Each approach leverages different types of information and comes with its own advantages and limitations.

challenge 1: Despite substantial advancements, recommender systems still face critical challenges rooted in the nature of user-item interaction data, including the cold-start problem, data sparsity, and imbalance **li2024recent, rama2019deep**. These limitations reduce prediction accuracy, preventing the system from effectively tailoring recommendations to user preferences.

challenge 2: Many existing models assume a static dataset, whereas in real-world scenarios, data arrives continuously in a streaming manner. The ability to handle such dynamic data influx remains a pressing challenge, as most traditional approaches lack adaptability to evolving user-item interactions.

To address these limitations, this paper proposes a Bayesian clustering framework for streaming recommendation that addresses sparsity and cold-start by leveraging user-side information. we derive an update rule with provable convergence guarantees, ensuring that the model adapts stably as the data stream evolves. This model enhances both the accuracy and adaptability of recommender systems.

The remainder of this paper is organized as follows. Section 2 reviews the literature on content-based, collaborative methods, with a detailed focus on clustering-based CF and the use of side information. Section 3 details the proposed method, including the clustering techniques employed and their integration into the recommender system. Section 4 presents the evaluation of the system’s performance, offering a comparative analysis with existing methods. Finally, Section 5 concludes the study and suggesting directions for future research.

2 Related Work

Methods in recommender systems are included three main categories content-based methods, collaborative Filtering(CF), and Hybrid methods **aljunid2025collaborative**. Content-based methods generate recommendations solely based on a user’s past behavior and interests, without considering other users’ interactions. While simple to implement, they provide a limited selection of similar items and fail to leverage relationships between users.

Collaborative filtering focuses on user-item interactions rather than item characteristics, recommending items based on the preferences of similar users. This approach enables more diverse and personalized recommendations by utilizing shared user behaviors and preferences **isinkaye2023matrix**. Collaborative methods have two types of algorithms, memory-based and model-based.

In memory-based methods, which are also called neighborhood-based methods, the relationship between users and items is investigated. User-based **zhao2010user**, **valcarce2019collaborative** or item-based **sarwar2001item**, **fu2018attention** similarity is considered to predict the user’s rating of items that he has not seen before. The primary limitation of memory-based methods is their reliance on loading the entire rating matrix into memory for each prediction, making them inefficient for large-scale datasets in terms of time and memory usage.

Model-based methods address this issue by employing a predictive model based on past user-item interactions. Various machine learning algorithms can be utilized in model-based approaches to enhance recommendation accuracy and efficiency. These mainly includes approaches like Matrix factorization **isinkaye2023matrix**, **shi2020user**, **yang2018enhancing**, **behera2023mixed**, Bayes classifier **valdiviezo2019collaborative**, neural networks **wang2015collaborative**, **angel2022deep**, **li2017collaborative**. This paper focuses on common challenges in recommender systems and clustering algorithms. Clustering techniques are commonly used to mitigate the sparsity problem and improve the scalability of CF methods. By grouping users or items with similar characteristics, clustering-based CF systems enhance recommendation performance while reducing computational complexity.

a) Clustering-based Collaborative Filtering

Clustering algorithms such as k-means **dakhel2011new**, **zahra2015novel**, **ranjan2018category**, **kant2017leaderrank** and FCM **birtolo2013advances** are widely utilized in recommender systems. Studies have explored various user clustering approaches, including Bayesian matrix factorization **bobadilla2018recommender**, evolutionary clustering that adapts user states over time **liji2018improved**, and weighted clustering that assigns higher importance to users with more interactions **salah2015dynamic**. In **katarya2018recommender**, a hybrid method used gray wolf optimization for initializing clusters, followed by fuzzy c-means (FCM) for user classification based on rating similarity . Similarly, for item clustering, in **deng2019novel**, an improved Kullback-Leibler divergence is employed to measure item similarity based on probability distributions. In **kant2017leaderrank**, the k-means algorithm is utilized to cluster items, and an item’s score is predicted solely based on the average scores of other items within the same cluster. Several methods have been proposed for clustering both users and items. Other approaches include power fuzzy clustering using an exponential fitness function to refine membership **treerattanapitak2012exponential** and network-based clustering where evolving statuses lead to stable groups **chen2017evolutionary**. Clustering via the QUBIC (A Qualitative Biclustering algorithm) in **singh2018impact**, aim to group similar users and items effectively. **science2018dynamic**

constructs a network that integrates user characteristics, item features, and temporal information, subsequently applying a dynamic evolutionary clustering algorithm.

Several advanced clustering-aware approaches such as **rajput2025nsga**, proposes an NSGA-II-optimized deep autoencoder framework improve multi-criteria recommendations by optimizing model parameters through a multi-objective approach, addressing data sparsity and non-linearity. **li2025self** presents a self-supervised graph Transformer (SGTN) that improves user-item representations by encoding ratings and applying contrastive learning. It focuses on capturing social and collaborative relationships through decentralized graphs and edge-aware attention. Although not explicitly designed for clustering, it enhances user embeddings, enabling dynamic and context-sensitive user-item grouping in recommendation systems. In **qian2025ifm** a framework that uses adversarial features from multiple models is proposed to improve transferability and enhance user-item representations. It refines embeddings by reducing noise and improving cluster separability, offering a strategy for robust clustering in dynamic recommendation systems. **he2025fedai** presents FedAI, a federated recommendation system that combines spectral clustering and K-means to group users while ensuring privacy. High-order interactions enhance embeddings, improving clustering in decentralized settings without accessing raw data.

b) Clustering based on User-side Information

While clustering-based methods address scalability and sparsity issues in the score matrix, the high dimensionality of user-item data makes them computationally expensive. To mitigate this, many approaches leverage side information as clustering features, significantly reducing the feature space and improving efficiency. **liji2018improved** employs user attributes such as age, gender, and residence to calculate feature distances for clustering. Similarly, **chen2019n2vscdnr** forms a bipartite user-item graph and converts each node into a vector before clustering. Additionally, **zhao2017coarse** clusters users based on social network labels and items via a bipartite graph, **sheugh2018novel** creates a two-dimensional similarity graph of users for clustering, and **wasid2018clustering** incorporates social information alongside the score matrix to enhance user clustering. Gong et al. **gong2010collaborative** leveraged both explicit ratings and implicit user behavior to improve similarity measurements. Lu et al. **lu2021novel** addressed the sparsity problem in user-item interactions by grouping items with similar features, thereby aligning recommendations more closely with user interests. Additionally, Wasid et al. **wasid2018improved** refined traditional collaborative filtering through a multi-criteria clustering approach, which integrates various aspects of user preferences to yield more diverse and accurate recommendations. Additionally, Mahalanobis distance is employed to measure the similarity between users' score vectors, taking into account not only the given scores but also their variance and covariance **deng2019neural** used variational autoencoders and side information to mitigate data sparsity and the cold start problem.

c) Recommendation in Streaming Data Environment

Furthermore, most of clustering-based methods often do not consider streaming data, making them less effective in dynamic environments. With the increasing need for real-time recommendations, online recommender systems have become essential for handling streaming data. These systems must efficiently update user preferences and item interactions as new information becomes available. Recommender systems in e-commerce and digital service platforms must continuously process new users and items, making real-time data handling essential. However, most existing systems rely on fixed datasets for evaluation, limiting their applicability in dynamic environments. Traditional content-based and memory-based collaborative filtering methods require loading all data into memory, making them unsuitable for streaming data. In contrast, model-based collaborative filtering, particularly matrix decomposition methods like ALS and SGD, allows incremental updates. While these methods address some streaming challenges, they remain computationally expensive and risk losing information during model updates. Clustering-based recommender systems for streaming data mainly rely on heuristics without leveraging prior knowledge, limiting their effectiveness. For instance, distance-based clustering adjusts noisy data positioning **georgieva2008dynamic**, as well as a density-based method updates clusters dynamically using local averages **baruah2012evolving**. In **das2007google** an online model proposed for Google News, continuously updates recommendations based on user interactions,

ensuring that personalized news suggestions remain relevant. A streaming matrix-factorization framework proposed in **jakomin2020simultaneous** that incrementally fuses multiple heterogeneous data streams to continuously adapt latent user-item representations and mitigate both data sparsity and cold-start issues. While online recommender systems rely on user feedback, new users, and new items for updates, many clustering-based methods struggle with streaming data. Integrating online learning into clustering-based collaborative filtering can improve adaptability in dynamic environments.

One of the primary strategies for mitigating data sparsity is cluster-based collaborative filtering. Clustering-based recommender systems can enhance recommendation reliability while maintaining lower computational complexity. Despite the benefits of clustering-based CF methods, challenges remain. Feature selection for clustering is a critical issue, as high-dimensional user-item matrices can lead to the curse of dimensionality. Conversely, clustering based on side features alone may not accurately capture user interests, reducing prediction accuracy. In addition, deal with streaming data, making them effective in dynamic environments is rally important.

To address these limitations, a Bayesian clustering model is introduced, incorporating prior knowledge for streaming data clustering. Unlike traditional clustering approaches, this method dynamically updates user and item clusters as new interactions occur. By combining user ratings with side features, the model improves personalization, incorporates real-time updates to handle user feedback, new users, and alleviates the cold start problem by leveraging additional user information beyond the rating matrix. Experimental results demonstrate that this approach outperforms existing clustering-based CF models in terms of accuracy and efficiency, making it well-suited for real-time recommendation systems.

3 Proposed Method

3.1 Overview: Bayesian Latent Filtering for Recommendation

The proposed method is built on a Bayesian latent filtering view of recommendation. Instead of directly assigning an incoming entity x to a recommendation set, the model first maps x into a latent representation, filters this representation through hidden components, and then decodes the activated hidden components into set-level recommendation evidence. The resulting inference chain is

$$x \longrightarrow \mathbf{z}(x) = \phi(x) \longrightarrow P(c_i | x) \longrightarrow f(\theta_j | c_i) \longrightarrow \mathcal{S}_j.$$

Here, x may denote an incoming user, item, or user-item representation, depending on the recommendation scenario. The variable c_i denotes the i -th hidden component, while $\mathcal{S}_j$ denotes the j -th candidate recommendation set. The descriptor θ_j characterizes the latent distribution of $\mathcal{S}_j$.

This formulation is not a conventional cluster-then-recommend strategy. In a hard clustering recommender, x is assigned to one cluster and the recommendation is obtained from that cluster only. In contrast, the proposed method marginalizes over all hidden components. Therefore, uncertainty about the latent state of x is preserved until the final recommendation stage.

3.2 Latent Filtering Formulation

Let

$$\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_K, \quad \mathcal{S}_j \subseteq \mathcal{X},$$

be candidate recommendation sets. Each set $\mathcal{S}_j$ is represented by a latent descriptor

$$\theta_j = (\mu_j, \Sigma_j),$$

where $\mu_j \in \mathbb{R}^d$ is the latent center of the set and $\Sigma_j \in \mathbb{R}^{d \times d}$ is its covariance matrix.

The incoming entity $x \in \mathcal{X}$ is first encoded into a latent vector

$$\mathbf{z}(x) = \phi(x) \in \mathbb{R}^d,$$

where $\phi : \mathcal{X} \rightarrow \mathbb{R}^d$ is an encoder. The encoder can be a fixed feature map, a dimensionality-reduction map, a matrix-factorization embedding, or a neural encoder. The proposed probabilistic formulation does not

require a specific choice of ϕ ; it only requires that x be represented in a latent space where Bayesian filtering can be performed.

We introduce C hidden components

$$c_i, \quad i = 1, \dots, C.$$

Each hidden component c_i is represented by a prototype and covariance

$$\psi_i = (\mathbf{m}_i, \Lambda_i),$$

where $\mathbf{m}_i \in \mathbb{R}^d$ is the prototype of the hidden component and Λ_i is its covariance. Thus, c_i is the hidden event, while $\mathbf{m}_i$ is its continuous representative in the latent space.

The likelihood of the encoded input under hidden component c_i is modeled as

$$p(\mathbf{z}(x) \mid c_i) = \mathcal{N}(\mathbf{z}(x); \mathbf{m}_i, \Lambda_i).$$

Therefore,

$$p(\mathbf{z}(x) \mid c_i) = \frac{1}{(2\pi)^{d/2} |\Lambda_i|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{z}(x) - \mathbf{m}_i)^T \Lambda_i^{-1} (\mathbf{z}(x) - \mathbf{m}_i) \right). \quad (1)$$

By Bayes' rule, the posterior activation of component c_i for input x is

$$q_i(x) \triangleq P(c_i \mid x) = \frac{\rho_i \mathcal{N}(\mathbf{z}(x); \mathbf{m}_i, \Lambda_i)}{\sum_{\ell=1}^C \rho_\ell \mathcal{N}(\mathbf{z}(x); \mathbf{m}_\ell, \Lambda_\ell)}, \quad (2)$$

where

$$\rho_i = P(c_i), \quad \rho_i \geq 0, \quad \sum_{i=1}^C \rho_i = 1.$$

The quantity $q_i(x)$ is the Bayesian latent filter. It measures how strongly the incoming entity x activates the hidden component c_i .

3.3 Set-Level Bayesian Decoder

After latent filtering, the model evaluates how strongly each hidden component supports a candidate recommendation set. Since c_i is represented by its prototype $\mathbf{m}_i$, the likelihood of the prototype under the descriptor of set $\mathcal{S}_j$ is defined as

$$f(\mathbf{m}_i \mid \theta_j) = \mathcal{N}(\mathbf{m}_i; \mu_j, \Sigma_j). \quad (3)$$

The explicit Gaussian form is

$$f(\mathbf{m}_i \mid \theta_j) = \frac{1}{(2\pi)^{d/2} |\Sigma_j|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{m}_i - \mu_j)^T \Sigma_j^{-1} (\mathbf{m}_i - \mu_j) \right). \quad (4)$$

The negative sign in the exponential is essential; otherwise the expression is not a valid Gaussian density.

Using Bayes' rule, the posterior association between the set descriptor θ_j and the hidden component c_i is

$$f(\theta_j \mid c_i) \equiv f(\theta_j \mid \mathbf{m}_i) = \frac{f(\mathbf{m}_i \mid \theta_j) P(\theta_j)}{\sum_{r=1}^K f(\mathbf{m}_i \mid \theta_r) P(\theta_r)}. \quad (5)$$

Here, $P(\theta_j)$ is the prior probability of the candidate set descriptor. It may be uniform when no prior preference over sets is available, or it may be estimated from historical popularity, set size, or temporal frequency in streaming data.

The relevance score of x to the candidate set $\mathcal{S}_j$ is obtained by marginalizing over all hidden components:

$$R_j(x) = \sum_{i=1}^C P(c_i \mid x) f(\theta_j \mid c_i). \quad (6)$$

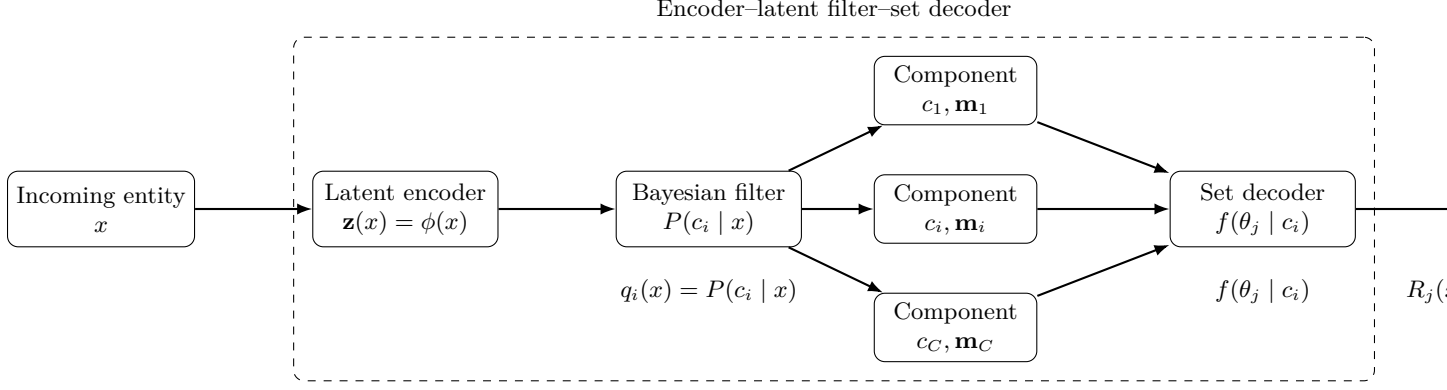


Figure 1: Bayesian latent filtering view of the proposed recommender. The structure is encoder-like because the input x is first mapped into a latent representation and then filtered through hidden components. It is not a classical autoencoder, because the objective is not to reconstruct x , but to infer the most relevant recommendation set $\hat{\mathcal{S}}(x)$.

The estimated recommendation set is therefore

$$\hat{\mathcal{S}}(x) = \mathcal{S}_{\hat{j}}, \quad \hat{j} = \arg \max_j R_j(x). \quad (7)$$

Equation (6) is the central probabilistic mechanism of the proposed method. The term $P(c_i | x)$ explains how the input activates the internal latent components, while $f(\theta_j | c_i)$ explains how each activated component supports a candidate recommendation set.

3.4 Encoder–Latent Filter–Set Decoder Structure

3.5 Statistical Justification

The latent component c_i is not introduced merely as a computational device. It provides a probabilistic factorization of the recommendation decision. Let Y be the set-relevance decision variable:

$$Y = j \iff x \text{ is relevant to } \mathcal{S}_j.$$

By the law of total probability,

$$P(Y = j | x) = \sum_{i=1}^C P(Y = j, c_i | x). \quad (8)$$

Using the product rule,

$$P(Y = j | x) = \sum_{i=1}^C P(Y = j | c_i, x) P(c_i | x). \quad (9)$$

We assume that the latent component c_i provides a sufficient intermediate representation of x for set-level inference:

$$P(Y = j | c_i, x) \simeq P(Y = j | c_i). \quad (10)$$

Therefore,

$$P(Y = j | x) \simeq \sum_{i=1}^C P(Y = j | c_i) P(c_i | x). \quad (11)$$

Identifying

$$P(Y = j | c_i) \equiv f(\theta_j | c_i),$$

we recover Eq. (6):

$$P(Y = j | x) \simeq R_j(x) = \sum_{i=1}^C P(c_i | x) f(\theta_j | c_i).$$

Proposition 1 (Bayesian optimality of the marginalized decision rule). *Under the conditional sufficiency assumption in Eq. (10), the decision rule*

$$\hat{j} = \arg \max_j R_j(x)$$

is the maximum a posteriori set estimator.

Proof. From Eqs. (9)–(11),

$$P(Y = j | x) \simeq \sum_{i=1}^C P(Y = j | c_i) P(c_i | x).$$

Since $P(Y = j | c_i) \equiv f(\theta_j | c_i)$, we have

$$P(Y = j | x) \simeq \sum_{i=1}^C P(c_i | x) f(\theta_j | c_i) = R_j(x).$$

Thus,

$$\arg \max_j P(Y = j | x) \simeq \arg \max_j R_j(x),$$

which proves that the proposed rule is the MAP estimator of the relevant set. $\square$

3.6 Learning the Latent Components and Set Descriptors

The latent components and set descriptors must be estimated from data. Let

$$\mathcal{B}_t = \{x_1^{(t)}, x_2^{(t)}, \dots, x_{n_t}^{(t)}\}$$

be the incoming mini-batch at streaming step t . For each sample, we compute

$$\mathbf{z}_n^{(t)} = \phi(x_n^{(t)}).$$

The posterior responsibility of component c_i for sample $x_n^{(t)}$ is

$$q_{ni}^{(t)} = P(c_i | x_n^{(t)}) = \frac{\rho_i^{(t)} \mathcal{N}(\mathbf{z}_n^{(t)}; \mathbf{m}_i^{(t)}, \Lambda_i^{(t)})}{\sum_{\ell=1}^C \rho_\ell^{(t)} \mathcal{N}(\mathbf{z}_n^{(t)}; \mathbf{m}_\ell^{(t)}, \Lambda_\ell^{(t)})}. \quad (12)$$

The effective mass of component c_i in the new mini-batch is

$$M_i^{(t)} = \sum_{n=1}^{n_t} q_{ni}^{(t)}. \quad (13)$$

With a forgetting factor $\lambda \in (0, 1]$, the streaming component mass is updated as

$$N_i^{(t+1)} = \lambda N_i^{(t)} + M_i^{(t)}. \quad (14)$$

The prior probability of component c_i is then updated by

$$\rho_i^{(t+1)} = \frac{N_i^{(t+1)}}{\sum_{\ell=1}^C N_\ell^{(t+1)}}. \quad (15)$$

The component prototype is updated as

$$\mathbf{m}_i^{(t+1)} = \frac{\lambda N_i^{(t)} \mathbf{m}_i^{(t)} + \sum_{n=1}^{n_t} q_{ni}^{(t)} \mathbf{z}_n^{(t)}}{N_i^{(t+1)} + \varepsilon}, \quad (16)$$

where $\varepsilon > 0$ prevents numerical instability.

The covariance of component c_i is updated by

$$\Lambda_i^{(t+1)} = \frac{\lambda N_i^{(t)} \Lambda_i^{(t)} + \sum_{n=1}^{n_t} q_{ni}^{(t)} \left(\mathbf{z}_n^{(t)} - \mathbf{m}_i^{(t+1)} \right) \left(\mathbf{z}_n^{(t)} - \mathbf{m}_i^{(t+1)} \right)^T}{N_i^{(t+1)} + \varepsilon} + \delta I, \quad (17)$$

where δI is a small diagonal regularizer that guarantees positive definiteness.

The descriptor of each candidate set $\mathcal{S}_j$ is computed from the latent evidence of its members. Let $x \in \mathcal{S}_j$ denote an entity belonging to set $\mathcal{S}_j$. The set mean is estimated as

$$\mu_j = \frac{\sum_{x \in \mathcal{S}_j} \sum_{i=1}^C P(c_i | x) \mathbf{m}_i}{\sum_{x \in \mathcal{S}_j} \sum_{i=1}^C P(c_i | x) + \varepsilon}. \quad (18)$$

The set covariance is estimated as

$$\Sigma_j = \frac{\sum_{x \in \mathcal{S}_j} \sum_{i=1}^C P(c_i | x) (\mathbf{m}_i - \mu_j)(\mathbf{m}_i - \mu_j)^T}{\sum_{x \in \mathcal{S}_j} \sum_{i=1}^C P(c_i | x) + \varepsilon} + \delta I. \quad (19)$$

Equations (18) and (19) show how the set descriptor $\theta_j = (\mu_j, \Sigma_j)$ is obtained. Therefore, $\mathcal{S}_j$ is not treated as a vague label; it is represented by a probability distribution in the latent space.

3.7 Latent-Filtered Rating Prediction

For recommendation, the set-level relevance score must be used directly in the prediction stage. Otherwise, the method would collapse into a conventional cluster-average recommender. We therefore define a normalized set relevance weight:

$$\alpha_j(x) = \frac{R_j(x)}{\sum_{\ell=1}^K R_\ell(x) + \varepsilon}. \quad (20)$$

Because each set descriptor has its own uncertainty, we define a confidence factor

$$\omega_j = \frac{1}{1 + \text{tr}(\Sigma_j)}. \quad (21)$$

A compact set descriptor produces a larger confidence value, while an uncertain descriptor is automatically down-weighted.

The final uncertainty-aware gating coefficient is

$$\gamma_j(x) = \frac{\alpha_j(x) \omega_j}{\sum_{\ell=1}^K \alpha_\ell(x) \omega_\ell + \varepsilon}. \quad (22)$$

Let u be the target user and a a candidate item. In the user-based collaborative filtering instantiation, $x = x_u$ is the representation of user u , and $\mathcal{S}_j$ is a candidate user set. For each set $\mathcal{S}_j$, define

$$\mathcal{N}_j(u, a) = \{v : v \in \mathcal{S}_j, r_{va} \text{ is observed}\}.$$

The local prediction contributed by set $\mathcal{S}_j$ is

$$\hat{r}_{ua}^{(j)} = \frac{\sum_{v \in \mathcal{N}_j(u, a)} s_j(u, v) r_{va}}{\sum_{v \in \mathcal{N}_j(u, a)} s_j(u, v) + \varepsilon}. \quad (23)$$

The within-set latent similarity is defined as

$$s_j(u, v) = \exp \left(-\frac{1}{2} (\mathbf{z}_u - \mathbf{z}_v)^T \Sigma_j^{-1} (\mathbf{z}_u - \mathbf{z}_v) \right), \quad (24)$$

where

$$\mathbf{z}_u = \phi(x_u), \quad \mathbf{z}_v = \phi(x_v).$$

The final predicted rating is obtained by marginalizing over all candidate sets:

$$\hat{r}_{ua} = b_u + b_a + \sum_{j=1}^K \gamma_j(x_u) \hat{r}_{ua}^{(j)}. \quad (25)$$

Here, b_u and b_a are optional user and item bias terms. If bias correction is not used, they are set to zero.

This prediction rule is fundamentally different from a simple cluster-average predictor. A cluster-average predictor first selects a cluster and then averages ratings inside that cluster. In contrast, Eq. (25) uses the full posterior chain

$$x_u \longrightarrow P(c_i | x_u) \longrightarrow f(\theta_j | c_i) \longrightarrow R_j(x_u) \longrightarrow \gamma_j(x_u) \longrightarrow \hat{r}_{ua}.$$

Thus, the final rating is an uncertainty-aware Bayesian mixture over recommendation sets.

For top- N recommendation, the predicted rating is computed for every unrated item $a \in \mathcal{I} \setminus \mathcal{I}_u$, and the N items with the largest predicted scores are returned:

$$\text{TopN}(u) = \{a_1, \dots, a_N : \hat{r}_{ua_1} \geq \hat{r}_{ua_2} \geq \dots \geq \hat{r}_{ua_N}\}, \quad (26)$$

where

$$a_k \in \mathcal{I} \setminus \mathcal{I}_u.$$

3.8 Streaming Algorithm

The complete streaming procedure is summarized in Algorithm 1. The algorithm receives data sequentially, updates the latent components through Bayesian responsibilities, updates the set descriptors, and then performs set-level or rating-level recommendation.

Algorithm 1 Bayesian Latent Filtering for Streaming Recommendation

Require: Streaming mini-batches $\{\mathcal{B}_t\}_{t=1}^T$, number of hidden components C , candidate sets $\{\mathcal{S}_j\}_{j=1}^K$, encoder ϕ , forgetting factor λ , regularizers ε, δ .

Ensure: Updated latent components $\{c_i\}_{i=1}^C$, set descriptors $\{\theta_j\}_{j=1}^K$, relevance scores $R_j(x)$, and recommendations.

- 1: Initialize $\rho_i^{(0)}$, $\mathbf{m}_i^{(0)}$, $\Lambda_i^{(0)}$, and $N_i^{(0)}$ for all $i = 1, \dots, C$.
- 2: Initialize $\theta_j^{(0)} = (\mu_j^{(0)}, \Sigma_j^{(0)})$ for all $j = 1, \dots, K$.
- 3: **for** $t = 1, \dots, T$ **do**

- 4: Receive mini-batch $\mathcal{B}_t = \{x_1^{(t)}, \dots, x_{n_t}^{(t)}\}$.

- 5: **for** each $x_n^{(t)} \in \mathcal{B}_t$ **do**

- 6: Compute latent representation:

$$\mathbf{z}_n^{(t)} = \phi(x_n^{(t)}).$$

- 7: Compute posterior component responsibilities:

$$q_{ni}^{(t)} = P(c_i \mid x_n^{(t)})$$

 using Eq. (12).

- 8: **end for**

- 9: **for** each hidden component c_i **do**

- 10: Update effective mass $N_i^{(t+1)}$ using Eq. (14).

- 11: Update prior $\rho_i^{(t+1)}$ using Eq. (15).

- 12: Update prototype $\mathbf{m}_i^{(t+1)}$ using Eq. (16).

- 13: Update covariance $\Lambda_i^{(t+1)}$ using Eq. (17).

- 14: **end for**

- 15: **for** each candidate set $\mathcal{S}_j$ **do**

- 16: Update set descriptor $\theta_j = (\mu_j, \Sigma_j)$ using Eqs. (18)–(19).

- 17: **end for**

- 18: **for** each query entity x **do**

- 19: Compute $P(c_i \mid x)$ for all hidden components.

- 20: Compute $f(\theta_j \mid c_i)$ for all candidate sets.

- 21: Compute $R_j(x)$ using Eq. (6).

- 22: Estimate the most relevant set:

$$\hat{\mathcal{S}}(x) = \mathcal{S}_{\arg \max_j R_j(x)}.$$

- 23: **end for**

- 24: For rating prediction, compute $\hat{r}_{ua}$ using Eq. (25).

- 25: **end for**

3.9 Objective Function and Convergence

The latent filtering stage can be interpreted as an online maximum-likelihood estimation problem for a Gaussian mixture in the latent space. For a mini-batch $\mathcal{B}_t$, define the negative log-likelihood

$$\mathcal{L}_t = - \sum_{n=1}^{n_t} \log \left[\sum_{i=1}^C \rho_i \mathcal{N}(\mathbf{z}_n^{(t)}; \mathbf{m}_i, \Lambda_i) \right]. \quad (27)$$

The posterior responsibility in Eq. (12) is exactly the E-step responsibility of this latent-variable model. The updates in Eqs. (14)–(17) are streaming M-step updates with forgetting.

Theorem 1 (Monotone improvement of the latent filtering likelihood). *Assume that ϕ is fixed during one streaming update, all covariance matrices are regularized as*

$$\Lambda_i \succeq \delta I, \quad \delta > 0,$$

and the responsibilities $q_{ni}^{(t)}$ are computed by Eq. (12). Then each exact EM update of ρ_i , $\mathbf{m}_i$, and Λ_i for a fixed mini-batch $\mathcal{B}_t$ does not increase the negative log-likelihood $\mathcal{L}_t$. Equivalently,

$$\mathcal{L}_t^{(k+1)} \leq \mathcal{L}_t^{(k)}.$$

Proof. For fixed parameters, the responsibilities $q_{ni}^{(t)}$ form a valid posterior distribution over hidden components:

$$q_{ni}^{(t)} \geq 0, \quad \sum_{i=1}^C q_{ni}^{(t)} = 1.$$

Using Jensen’s inequality, the negative log-likelihood in Eq. (27) is upper-bounded by the standard EM free-energy functional. The E-step sets $q_{ni}^{(t)}$ equal to the posterior responsibilities, which makes the bound tight. The M-step minimizes this bound with respect to ρ_i , $\mathbf{m}_i$, and Λ_i . Therefore, the likelihood cannot decrease, or equivalently the negative log-likelihood cannot increase. The covariance regularization $\Lambda_i \succeq \delta I$ prevents singular covariance matrices and keeps the objective well-defined. $\square$

Corollary 1 (Convergence of the inner latent update). *If the latent representations $\mathbf{z}_n$ are bounded and covariance matrices are regularized by δI , then the sequence of negative log-likelihood values generated by the inner latent filtering update is monotonically non-increasing and bounded below. Therefore, it converges to a finite value.*

The theorem establishes convergence of the probabilistic latent filtering stage. This is stronger and cleaner than relying on a positive-definite Hessian argument for a fuzzy clustering objective. The proposed method is therefore supported by a standard latent-variable likelihood principle rather than by a purely heuristic clustering update.

3.10 Discussion: Why the Method Is Not Ordinary Clustering

The proposed model differs from ordinary clustering-based recommendation in three essential ways.

First, the assignment of x to hidden components is not hard. The model computes a posterior distribution

$$P(c_i | x),$$

which preserves uncertainty over all hidden components.

Second, the final recommendation set is not selected directly from a single cluster. Instead, set relevance is computed by marginalization:

$$R_j(x) = \sum_{i=1}^C P(c_i | x) f(\theta_j | c_i).$$

Thus, the hidden layer acts as a Bayesian mediator between incomplete observations and recommendation-set inference.

Third, rating prediction is not obtained by a simple average inside a cluster. The final prediction uses the uncertainty-aware gate

$$\gamma_j(x)$$

and aggregates local predictions across multiple candidate sets:

$$\hat{r}_{ua} = b_u + b_a + \sum_{j=1}^K \gamma_j(x_u) \hat{r}_{ua}^{(j)}.$$

Therefore, the recommendation process remains consistent with the Bayesian latent filtering formulation from the input representation to the final top- N output.

“latex

4 Experiments

This section evaluates the proposed Bayesian latent-filtering recommender from four complementary perspectives: (i) rating prediction accuracy, (ii) top- N decision-support quality, (iii) streaming computational overhead, and (iv) stability of the latent filtering update. The purpose of the experiments is not to present the method as a standard FCM variant, but to examine whether the proposed posterior chain

$$x \longrightarrow P(c_i | x) \longrightarrow f(\theta_j | c_i) \longrightarrow R_j(x) \longrightarrow \hat{r}_{ua}$$

improves recommendation quality under sparsity, cold-start, and streaming conditions.

4.1 Datasets

The proposed recommender system was evaluated on three benchmark datasets: MovieLens-100K, MovieLens-1M, and Yelp. MovieLens-100K contains 100,000 ratings from approximately 1,000 users on 1,700 movies. MovieLens-1M contains about 1 million ratings from approximately 6,000 users on 4,000 movies. Both MovieLens datasets include movie-side information such as genres and user-side attributes such as age, gender, and occupation.

The Yelp dataset contains user reviews and business information. In this study, we focus on the restaurant category. Business-side attributes, such as parking, child-friendliness, and ambiance, are transformed into numerical features. Available categorical attributes are encoded into binary or discrete numerical values. Missing or unspecified attributes are represented by a reserved value.

In all datasets, the observed rating matrix is sparse. Therefore, the experiments are designed to evaluate not only prediction accuracy, but also the ability of the proposed latent filtering mechanism to exploit side information and streaming evidence when user-item interactions are incomplete.

4.2 Streaming Evaluation Protocol

To reflect a streaming recommendation scenario, the training data is divided into an initialization block and a sequence of incoming mini-batches. The initialization block is used to estimate the initial latent components and set descriptors. After that, new users or user-item interactions arrive sequentially, and the model updates its posterior component activations and set descriptors without retraining the whole model from scratch.

For each incoming user representation x_u , the model computes

$$P(c_i | x_u)$$

using the Bayesian latent filter. Then it computes the set-level posterior evidence

$$R_j(x_u) = \sum_{i=1}^C P(c_i | x_u) f(\theta_j | c_i).$$

The normalized relevance weights are then used for rating prediction according to Eq. (25). This protocol ensures that the experimental procedure is consistent with the proposed probabilistic formulation.

4.3 Evaluation Metrics

We evaluate the method using two groups of metrics: rating prediction metrics and decision-support metrics.

Prediction accuracy is measured by Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

$$\text{MAE} = \frac{1}{|\Omega_{\text{test}}|} \sum_{(u,a) \in \Omega_{\text{test}}} |\hat{r}_{ua} - r_{ua}|, \quad (28)$$

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum_{(u,a) \in \Omega_{\text{test}}} (\hat{r}_{ua} - r_{ua})^2}, \quad (29)$$

where Ω_{test} denotes the set of test user-item pairs, r_{ua} is the true rating, and $\hat{r}_{ua}$ is the predicted rating.

For decision-support evaluation, items with ratings greater than or equal to 4 are considered relevant, while items with ratings less than or equal to 3 are considered non-relevant. Precision, recall, and F1 score are defined as

$$\begin{aligned}\text{Precision} &= \frac{\#tp}{\#tp + \#fp}, \\ \text{Recall} &= \frac{\#tp}{\#tp + \#fn}, \\ F1 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.\end{aligned}$$

Here, $\#tp$, $\#fp$, and $\#fn$ denote true positives, false positives, and false negatives, respectively.

4.4 Comparison with Clustering-Aware Recommendation Baselines

Table 1 compares the proposed method with representative clustering-aware and side-information-based recommendation baselines. These baselines are selected because they are closest to the proposed method in terms of using clustering, user-side information, or neighborhood-based prediction under sparse observations.

Table 1: Comparison of the proposed Bayesian latent-filtering recommender with clustering-aware recommendation baselines.

Dataset	Method	MAE	RMSE	Precision	Recall	F1
ML_100K	Chen liji2018improved	0.74	0.95	0.88	0.50	0.642
	Kant kant2017leaderrank	0.75	0.95	0.56	0.87	0.681
	Enayatifar mohammadpour2019efficient	0.78	0.96	–	–	–
	Wasid wasid2018clustering	0.91	1.12	0.54	0.75	0.627
	Proposed method	0.71	0.91	0.56	0.89	0.687
ML_1M	Chen liji2018improved	0.72	0.94	0.853	0.49	0.622
	Kant kant2017leaderrank	0.72	0.925	0.53	0.86	0.653
	Enayatifar mohammadpour2019efficient	0.85	1.35	–	–	–
	Wasid wasid2018clustering	0.95	1.24	0.56	0.71	0.626
	Proposed method	0.705	0.925	0.62	0.85	0.710
Yelp	Kant kant2017leaderrank	0.965	1.25	0.57	0.78	0.650
	Proposed method	0.943	1.21	0.61	0.81	0.690

The results show that the proposed method achieves the lowest MAE on all three datasets and improves F1 score over the reported baselines. The improvement is especially relevant under sparse observations because the proposed method does not rely on a hard cluster assignment. Instead, it keeps the posterior evidence over multiple latent components and candidate sets until the final prediction stage.

It should be emphasized that Table 1 compares against clustering-aware baselines rather than against the entire modern recommender-system literature. Therefore, the table supports the claim that the proposed Bayesian latent-filtering mechanism improves over closely related clustering-based approaches. It should not be interpreted as a universal state-of-the-art comparison against all neural, graph-based, or large-scale recommender models.

4.5 Computational Overhead in Streaming Prediction

Table 2 reports the maximum computational time per prediction. The proposed method introduces a small additional overhead compared with Kant et al. **kant2017leaderrank**, because it computes posterior latent

activations and uncertainty-aware set relevance weights. However, the additional cost remains limited in all three datasets.

Table 2: Maximum computational time per prediction in milliseconds.

Method	Yelp	MovieLens-100K	MovieLens-1M
Kant et al. kant2017leaderrank	48 ms	18 ms	25 ms
Proposed method	50 ms	22 ms	28 ms

These results should be interpreted carefully. The proposed method is not faster than Kant et al. in raw prediction time. Rather, it trades a small increase in computation time for improved rating accuracy and better posterior uncertainty modeling. This is consistent with the Bayesian latent-filtering design, where additional probabilistic evidence is computed before the final prediction.

4.6 Latent Filtering Stability and FCM Diagnostic

The comparison with standard FCM is used only as an optimization diagnostic, not as the main experimental claim of the paper. Standard FCM provides a useful reference because it performs soft assignment without Bayesian posterior filtering. In contrast, the proposed method incorporates prior evidence and posterior latent activations.

For this diagnostic experiment, the same latent representations and the same number of components are used for both methods. The objective value is tracked across iterations to compare the stability of the update process. Figure 2 shows the convergence behavior on MovieLens-100K, MovieLens-1M, and Yelp.

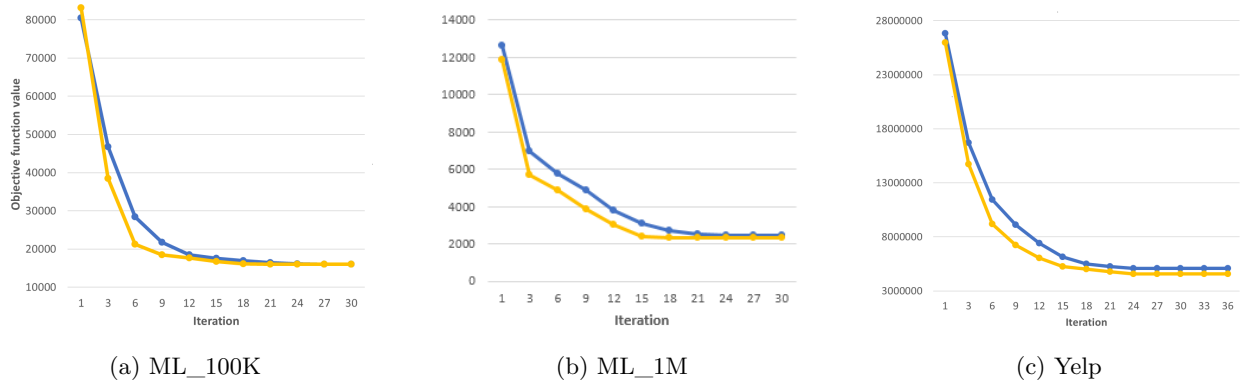


Figure 2: Diagnostic comparison of convergence behavior between standard FCM and the proposed Bayesian latent-filtering update. The comparison is used to evaluate update stability, not to define the proposed method as an FCM variant.

Table 3 reports the minimum objective value and the number of iterations required by the two update mechanisms. The proposed update reaches a lower diagnostic objective value and requires fewer iterations in all three datasets.

This diagnostic result supports the theoretical analysis of the latent update: incorporating Bayesian weighting and prior evidence improves the stability of the iterative update. However, the main recommendation performance should be judged by MAE, RMSE, precision, recall, and F1, not by the FCM objective alone.

4.7 Parameter Sensitivity

The proposed method depends on the number of latent components and the amount of data used for initialization. To study this effect, we evaluate different numbers of user-side latent components and two

Table 3: Diagnostic convergence comparison between standard FCM and the proposed Bayesian latent-filtering update.

Dataset	FCM		Proposed update	
	Value	Iterations	Value	Iterations
ML-100K	2709	18	2402	15
ML-1M	16099	22	16093	19
Yelp	5078242	26	4587557	23

initialization ratios: 30% and 40% of the training data. The remaining training stream is then processed sequentially.

Table 4: Prediction accuracy of the proposed method under different initialization ratios and numbers of latent components.

Dataset	Initial data	60		80		100		110	
		MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
ML_100K	30%	0.73	0.925	0.72	0.920	0.76	0.955	–	–
	40%	0.73	0.917	0.71	0.910	0.74	0.960	–	–
ML_1M	30%	0.78	1.000	0.763	0.979	0.720	0.950	0.725	0.935
	40%	0.77	0.980	0.752	0.976	0.705	0.925	0.718	0.930
Yelp	30%	0.993	1.450	0.975	1.430	0.965	1.310	0.955	1.320
	40%	0.990	1.470	0.965	1.400	0.945	1.250	0.943	1.210

The results show that increasing the initialization ratio from 30% to 40% generally improves prediction accuracy. This is expected because a larger initialization block provides more stable estimates of the latent components and set descriptors. The best number of latent components differs across datasets: MovieLens-100K performs best with 80 components, MovieLens-1M with 100 components, and Yelp with 110 components. This indicates that the appropriate latent resolution depends on dataset size, sparsity, and heterogeneity.

4.8 Experimental Summary

The experiments support three main observations. First, the proposed method improves MAE and F1 score over closely related clustering-aware recommendation baselines. Second, the Bayesian latent-filtering update introduces only a small computational overhead while preserving streaming adaptability. Third, the FCM comparison should be interpreted only as a diagnostic study of update stability, not as the core identity of the proposed recommender.

Overall, the experimental evidence is consistent with the theoretical formulation: the method improves recommendation quality by maintaining posterior uncertainty over latent components and recommendation sets, rather than reducing the decision to a hard cluster assignment followed by simple averaging. ““

5 Conclusion

In this study, a method based on the Bayesian clustering model is used. Initially, to reduce the feature for user clustering, the items are clustered using FCM. Then, As considering the users are not constant and added to the system dynamically, a Bayesian clustering method is used for user clustering. In this approach, learning is conducted in a distributed manner, where cluster centroids interact and influence each another based on their relative distances. This minimizes the impact of outliers on the cluster centroids. Finally,

after clustering, the scores of neighboring users are used to predict a user’s score on an item that they have not seen yet, and the items with the highest scores are suggested to each user. The evaluation results on streaming data events demonstrate that the proposed method outperforms existing approaches in enhancing recommender system performance, achieving more precise user score predictions. In proposed method, items are considered constant, which is a limitation in real-world application. Future research could address this by considering dynamic item. Furthermore, integrating a score prediction methods that utilize both user-user and item-item similarities could enhance recommendation accuracy.